Day 1: Intro to Embedded & Zephyr Toolchain

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

What is an Embedded System

An **embedded system** is a dedicated computer built into a larger device to perform a specific function — unlike a general-purpose PC. Examples: your nRF52840 inside a fitness tracker, a car ECU, or a smart thermostat. Embedded systems are constrained by limited RAM, Flash, CPU speed, and power budget, which is why learning the right C++ techniques matters so much.

nRF52840 SoC Overview

The **nRF52840** (by Nordic Semiconductor) is an ARM Cortex-M4F microcontroller with:

- **CPU:** 64 MHz ARM Cortex-M4 with FPU
- **Flash:** 1 MB (where your code lives)
- **RAM:** 256 KB (stack + heap + globals)
- **Peripherals:** BLE 5.0, USB, UART, SPI, I2C, I2S, QSPI, ADC, PWM, GPIOTE, RTC, etc.

The nRF52840DK development kit breaks out all pins and includes a J-Link debugger on-board.

Zephyr RTOS Architecture

Zephyr is a scalable, real-time operating system (RTOS) designed for constrained embedded devices. Key parts:

- **Kernel:** Scheduler, threads, semaphores, mutexes, message queues
- **Device drivers:** Standardized API for GPIO, UART, SPI, I2C, sensors, etc.
- **Subsystems:** Bluetooth, USB, networking, power management, logging

- **Build system:** CMake + Kconfig + Devicetree — all integrated via the `west` tool

West Meta-tool

`west` is Zephyr's command-line tool for managing everything:

```
west build -b nrf52840dk/nrf52840 .  # Compile for nRF52840DK
west flash                            # Flash firmware over J-Link
west debug                            # Start GDB debug session
west update                           # Sync all Zephyr modules
```

Think of `west` as the glue between CMake, the Zephyr SDK, and your hardware.

CMakeLists.txt & Kconfig

Every Zephyr app needs two files:

- **CMakeLists.txt** — tells CMake which source files to compile and links against the Zephyr kernel
- **prj.conf** — Kconfig options that enable/disable Zephyr subsystems (e.g., `CONFIG_GPIO=y`, `CONFIG_LOG=y`). These become compile-time `#define` values that control which code is included in the binary.

Devicetree (DTS) Overview

Devicetree describes the hardware layout of your board in a text format — which peripheral is on which pins, what bus speed, interrupt number, etc. Your C++ code references hardware by DTS **aliases** (`DT_ALIAS(led0)`) rather than hardcoded register addresses, making the code portable across different boards without recompilation.

Practice Tasks

1. Run `west init ~/zephyrproject && west update` to set up your workspace
2. Browse `zephyr/boards/arm/nrf52840dk_nrf52840/` to see the board DTS files

3. Open `build/zephyr/zephyr.map` after building — find where `main` is placed in Flash
4. Change `CONFIG_PRINTK=y` to `n` in `prj.conf`, rebuild — observe the difference

---

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day01
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach   # RTT viewer (requires J-Link)
```

Resources

- Zephyr Docs
- nRF52840 Product Specification
- Nordic DevZone
- Source: src/day01/main.cpp